

EasyRTG

An Intelligent No-Code Platform for Training Robots

Shivam Chavan
Student ID: 3053682

Supervisor: Shraddha Naik

MSc in Artificial Intelligence and Computer Science
School of Computer Science
University of Birmingham Dubai
August 2026

Declaration and Honour Code

I certify that this project is my own work. I defined the project aims, system requirements, architecture, feature set and experimental programme, and I take responsibility for the submitted software, evidence, interpretation and conclusions. Anthropic's Claude assisted with parts of application development and debugging. OpenAI Codex was used only during report preparation to refine and improve my original material for clarity and consistency and to improve formatting. I reviewed and edited all AI-assisted output before including it. The following page provides a detailed account of each tool's contribution.

Ideas, software and published findings from other authors are acknowledged through numbered citations and corresponding bibliography entries. I understand that I remain accountable for the accuracy of the report, the behaviour of the submitted system and compliance with the University's academic-integrity requirements.

Name: Shivam Chavan

Student ID: 3053682

Use of Generative AI

Two generative-AI tools were used: Anthropic's Claude during application development and OpenAI Codex during report preparation. Their roles were different and are declared separately below.

- **My contribution and ownership.** I selected the problem, defined the aims and requirements, designed the architecture and feature set, chose the experimental questions and made the final engineering decisions. I configured, launched and interpreted the experiments in Chapter 6. The reported values come from the retained run telemetry, and the conclusions reflect my assessment of that evidence. I also provided initial draft of the report to chatgpt for language and grammar refinement. Most sections of the report are refined to match the tone and language style.
- **Claude: desktop user interface.** I specified the screens, state flow and intended behaviour of each panel. Claude generated most of the resulting Dart code. I reviewed, executed, corrected and integrated it, and the submitted frontend contains substantial revisions. The desktop interface is therefore largely AI-generated under my direction.
- **Claude: backend implementation and debugging.** I designed the two simulation lanes, morphology layer, expression compiler and audit, assistant toolbox and approval gates described in Chapter 4. Claude drafted parts of the backend implementation beyond boilerplate and docstrings, helped interpret stack traces and suggested candidate fixes. I defined the required interfaces and behaviour and verified retained changes against the running system and test suite.
- **Claude: implementation options.** Claude compared approaches that I was already considering, including whether Isaac Sim should be embedded in the backend or launched as a subprocess. I made the final choices and developed the reasoning recorded in Chapter 5.
- **OpenAI GPT: report refinement and editing only.** I supplied the original report content, project evidence, experimental results, selected sources, interpretations and conclusions. Codex reiterated and refined the prose across the report to improve clarity, consistency, academic tone and formatting.

All AI-assisted material was reviewed against source code, Git history, test results, retained telemetry, recorded videos and cited literature as appropriate. I understand the submitted code and take responsibility for the final software and report.

Acknowledgements

I am sincerely grateful to my supervisor, Shraddha Naik, for her patience, guidance and constructive feedback throughout this project. She encouraged me to examine the relevant literature more carefully and to connect the implementation to established research rather than treating it only as a software project. Her broader knowledge of software development, academic writing and the qualities expected of a complete application helped me improve both the system and this report.

I would also like to thank the School of Computer Science and its teaching staff for the knowledge, guidance and learning opportunities they provided throughout my degree. The principles and skills developed during the programme formed an important foundation for this project.

Finally, I am grateful to my friends and colleagues for their encouragement during the less visible and more difficult parts of the project. Their support helped me persist through unsuccessful simulator configurations, policies that did not learn the intended behaviour and long-running experiments. These setbacks ultimately contributed to several of the design decisions documented in this report.

Abstract

Training a robot with reinforcement learning requires a delicate integration of a physics simulator, be it headless, a python environment, a reward function, a learning algorithm, monitoring and policy evaluation. Although existing frameworks provide these capabilities, setting them up normally requires simulator-specific pre-existing knowledge and programming experience. EasyRTG, my application, addresses this barrier through a no-code desktop workflow in which a user imports a robot in Universal Robot Description Format (URDF) and describes the required behaviour, reviews an assistant-generated environmental configuration and reward function, validates the reward, trains a policy and evaluates the result.

My system combines a Flutter desktop interface, a Fast API orchestration backend, PyBullet inspection of the imported robot and reward testing, and NVIDIA Isaac Sim training and playback. EasyRTG derives control-relevant robot structure directly from the URDF and represents rewards as well-crafted mathematical expressions. This removes fixed joint-name rules and predefined reward items, allowing my application pipeline to support quadruped and humanoid robots.

EasyRTG was run in 40+ simulation runs with few on the Unitree A1 robot, the ANYmal robot. The rest of the runs were on the Unitree Go1 robot, and the NASA Valkyrie robot both employing PPO, SAC and TD3. Of the 40 runs, 34 led to policies that were deemed useful. 12 out of those 34 analyzable cases, reached at least 95% of the 1000-step episode limit, with peak performance reaching around 32,000 environment steps per second on a single RTX 5000 GPU.

Best policies of PPO, SAC, and TD3 reached completion rates of 96.3%, 100%, and 99.1% respectively on the Go1, PPO, and SAC. The Valkyrie painted a different picture though, SAC was at 82.1%, PPO and TD3 around 20%. The selection of an algorithm is a much more significant factor.

The evaluation also shows that a high reward does not necessarily represent the intended robot behaviour. On the galloping task of GO1 robot, TD3 algorithm produced a fast trot without a true flight phase required for galloping. EasyRTG demonstrates a working integration of language-assisted configuration along with general robot inspection and accelerated learning.

Table of Contents

1 Introduction 12

1.1 Project motivation 12

1.2 Problem statement 12

1.3 Aim and objectives 13

1.4 Research questions 13

1.5 Scope and contributions 14

1.6 Organisation of this report 14

2 Background and Literature Survey 15

2.1 Reinforcement learning for robot control 15

2.2 The three learners: PPO, SAC and TD3 15

2.3 From single environments to massively parallel simulation 16

2.4 Isaac Gym and the GPU-native turn 16

2.5 Orbit and Isaac Lab 16

2.6 Learning to walk in minutes 16

2.7 Reward design as the real bottleneck 17

2.8 Eureka: language models writing rewards 17

2.9 DrEureka: pushing language-assisted design toward hardware 17

2.10 General-purpose simulators: Gazebo and Unity ML-Agents 18

2.11 Unity RL Playground: the closest accessibility claim 18

2.12 Where EasyRTG sits 18

3 Research Methodology and Requirements 19

3.1 Design-science approach 19

3.2 Evidence sources 19

3.3 Functional and non-functional requirements 20

3.4 Experimental protocol and metrics 20
3.5 Software verification 21
4 System Design and Architecture 21
4.1 Architectural overview 21
4.2 URDF inspection and morphology 22
4.3 Expression rewards and the shared state vocabulary 22
4.4 Assistant and human oversight 23
4.5 Training, monitoring and evaluation 24
5 Implementation and Iterative Development 25
5.1 From proposal to working system 25
5.2 Milestone history 25
5.3 Key engineering decisions 27
5.4 Security and reliability improvements 27
5.5 Role of generative AI in the development process 27
6 Experiments, Results and Findings 28
6.1 Run coverage and failure history 28
6.2 Fleet-level results 28
6.3 What the reward actually paid for 29
6.4 Throughput and the GPU lane 30
6.5 How long learning took 31
6.6 Go1: three learners, one solved robot 31
6.7 Go1 walking and galloping 32
6.8 Valkyrie: a controlled algorithm comparison 33
6.9 Functional verification 35
7 Discussion 36
7.1 Answering the research questions 36

7.2 What the project learned about abstraction 36
7.3 Limitations and threats to validity 37
7.4 Practical and academic significance 38
8 Conclusion and Future Work 38
8.1 Conclusion 38
8.2 Future work 39
Bibliography 40
Appendix A GitLab Repository and Running the Project 42
A.1 Repository 42
A.2 Minimum setup 42
A.3 Verification commands 42
Appendix B Experiment Coverage 43
Appendix C Supplementary Figures 43
Appendix D Full Run Index 46
Appendix E Compute Environment and Reproduction Notes 47

List of Figures

Figure 1. The EasyRTG workflow, from robot import to policy evaluation. 12
Figure 2. EasyRTG architecture, and the separation between the inspection and GPU training lanes. 22
Figure 3. The EasyRTG desktop assistant, showing a generated training configuration and the changes offered for review. 24
Figure 4. Major implementation and experiment milestones, reconstructed from Git dates and the run archive. 25
Figure 5. Episode survival for the best observed run in each robot and algorithm pairing. 29
Figure 6. Term-by-term decomposition of the reward at the peak-reward step of the best Go1 run. 30
Figure 7. Peak simulation throughput against the number of parallel environments, coloured by robot. 30
Figure 8. Reward for the best Go1 run under each learner, at 4,096 environments with the same reward expression. 32
Figure 9. The Unitree Go1 during recorded policy evaluation. 33
Figure 10. Reward for Valkyrie under SAC and TD3, same reward expression and same 40M-step budget. 34
Figure 11. Episode length reveals that SAC's advantage came from staying upright, not from scoring faster. 34
Figure 12. The NASA Valkyrie during recorded SAC policy evaluation. 35
Figure 13. Best mean episode length for every analysable Isaac run, in chronological order. 44
Figure 14. Reward against environment steps for the best run of each robot family.44
Figure 15. Environment steps before each run's smoothed mean episode length first holds above 900. 45
Figure 16. Episode length for the best Go1 run under each learner. 45

List of Tables

Table 1. Research questions used to organise the implementation and evaluation.14

Table 2. Algorithm rationale and its observed role in EasyRTG. 15

Table 3. Positioning EasyRTG against representative robot-simulation and learning platforms. 19

Table 4. Traceable project requirements. 20

Table 5. Engineering decisions that emerged through iteration. 27

Table 6. Best observed run for each robot and algorithm pairing. 28

Table 7. Coverage of the 34 analysable Isaac runs by robot and algorithm. 43

Table 8. All 34 analysable Isaac runs, 13 July to 18 August 2026. 46

Table 9. Compute and software environment used for all experiments. 47

Abbreviations

Term	Meaning
API	Application Programming Interface
AST	Abstract Syntax Tree
DRL	Deep Reinforcement Learning
GPU	Graphics Processing Unit
LLM	Large Language Model
PPO	Proximal Policy Optimisation
REST	Representational State Transfer
RL	Reinforcement Learning
SAC	Soft Actor-Critic
TD3	Twin Delayed Deep Deterministic Policy Gradient
UI	User Interface
URDF	Universal Robot Description Format
USD	Universal Scene Description

1 Introduction

1.1 Project motivation

Reinforcement learning is closed loop in which an agent observes the state of an environment its put into, selects an action form the available options, and receives a reward based on the selected action [1]. Applying this closed loop to robot control requires considerale work and understanding of tools and techniques. The user should first set up a simulator and read the robot model, then load the robot and create the observations and actions, design the reward function, set the training parameters, execute the training process, and judge the performance of policy after training.

Existing RL libraries simplify the method of usage as much as possible, but they usually assume that the user already has the knowledge to build a valid environment [2]. This requirement introduces a challenge for new users, a dependency of knowledge of each tool on each part of the workflow. Designing the reward is impossible without understanding and interpreting the state is impossible without understanding the robot and the simulator.

EasyRTG project motivated by a previous robotics project where the robot could learn the desired behavior, but by the time it truly learnt, a significant amount of manual work and time that had been done in integrating the simulator, the environment code, the reward function, the training scripts and the visualization. This underlying effort wasn't reflected in the final demonstration. EasyRTG was thus born, making environment creation and experiment management a product of the application and not an undocumented prerequisite.

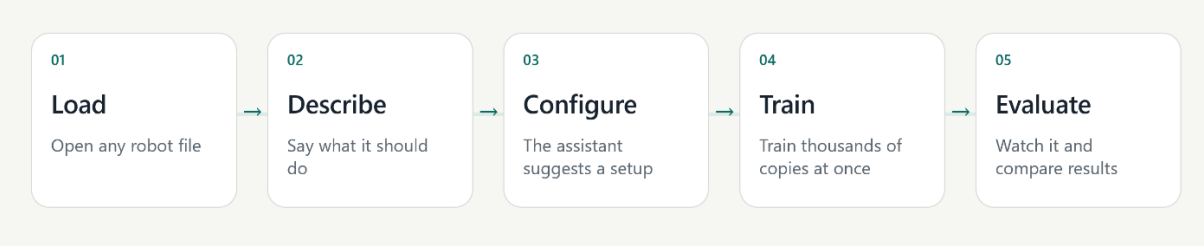


Figure 1. The EasyRTG workflow, from robot import to policy evaluation.

1.2 Problem statement

Currently, many different, unrelated skills must be implemented to convert a raw robot description and a specified task into a working reinforcement learning environment. Today's physics engines and RL toolkits can do it, but they're essentially low-level scripted and

configured, and they rely on convention that varies based on the simulator. A standard URDF file specifies physical geometry and the hierarchy of the joints, but it does not specify the most important learning parameters: which degrees of freedom to actuate, allowed contact links, reset distributions, reward formulation. The robot's physical structure and the desired movement needs to be manually translated into environmental code which ensures reliable training. In this project, it is explored if this translation can be achieved by an integrated and inspectable desktop GUI, without resorting to low-level simulator programming, while maintaining the control of the user.

1.3 Aim and objectives

The aim of this project was to develop a desktop platform that takes a user from a URDF robot and a natural-language objective to a trained and evaluated policy without requiring simulator or reinforcement-learning code. The objectives were to:

- Provide a single desktop workflow for robot inspection, configuration, monitoring, training and evaluation.
- Derive the morphology required for control directly from the URDF without relying on robot-specific joint-name rules.
- Allow an LLM assistant to propose reward expressions and configuration changes while keeping consequential actions reviewable and subject to user approval.
- Use GPU-parallel simulation to make repeated robot-learning experiments practical on a single workstation.
- Evaluate the platform across multiple robot bodies, tasks and learning algorithms using saved telemetry and recorded behaviour.

1.4 Research questions

The implementation and evaluation were guided by the following four research questions.

Table 1. Research questions used to organise the implementation and evaluation.

ID	Research question
RQ1	How can a no-code interface translate a URDF and a natural-language goal into a valid RL configuration while retaining human oversight?
RQ2	To what extent can morphology-derived configuration and expression rewards support different robot bodies without robot-specific application logic?
RQ3	How do PPO, SAC and TD3 differ across quadruped and humanoid locomotion experiments?
RQ4	How do parallel environment count and robot complexity affect GPU simulation throughput?

1.5 Scope and contributions

EasyRTG is a single user desktop robot simulator application for URDF robots. These contributions include an integrated workflow, name-based configuration system, expression-based rewards feature, manual configuration auditing, controlled ai assistant and retained run provenance. No new reinforcement-learning algorithm is proposed in the project.

The scope of my project does not include any physical deployment of the robot, terrain curricula, camera observation, ROS integration and multi-agent learning. Therefore, the evaluation only shows evidence of a platform that can configure and train multiple simulated legged robots. It does not give evidence of the transfer to the physical hardware or access to multiple users simultaneously.

1.6 Organisation of this report

Chapter 2 provides an overview of the reinforcement-learning and GPU-native simulation algorithms, reward-design research and related accessibility platforms used in this work. In the 3rd chapter, the design-science methodology, sources of evidence, requirements and experimental protocol are described. The system architecture and the main technical mechanisms are presented in Chapter 4. The iterative development process has been explained in Chapter 5. The experiments and results of my simulation runs are shown in Chapter 6. The research questions are answered along with the limitations of study in Chapter 7. The conclusion and future work is presented in chapter 8.

2 Background and Literature Survey

2.1 Reinforcement learning for robot control

In reinforcement learning, a policy selects actions from observations and is improved using rewards collected through interaction with an environment [1]. This formulation is suitable for robotics because the desired behaviour can be specified without providing a labelled action for every possible state. However, continuous action spaces, contact dynamics and long training horizons increase both learning instability and the cost of reward-design errors [2]. Robot policies are therefore normally trained in simulation before they are considered for physical deployment. All experiments in this report were conducted in simulation.

2.2 The three learners: PPO, SAC and TD3

Three algorithms were selected for the evaluation. Proximal Policy Optimisation (PPO) is an on-policy method that limits destructive policy updates through a clipped surrogate objective [3]. Its simplicity and reliable implementations have made it a common baseline for locomotion. Soft Actor-Critic (SAC) is an off-policy method that maximises expected return together with policy entropy [4], thereby maintaining exploration in continuous-control tasks. Twin Delayed Deep Deterministic Policy Gradient (TD3) addresses value overestimation through twin critics, delayed policy updates and target-policy smoothing [5].

EasyRTG initially supported PPO only. SAC and TD3 were added after the pipeline had demonstrated that a robot could learn, allowing the evaluation to examine whether different learners produced different outcomes under the same task configuration.

Table 2. Algorithm rationale and its observed role in EasyRTG.

Algorithm	Learning style	Reason for inclusion	Observed project role
PPO	On-policy, stochastic	Common locomotion baseline; stable implementation	Strong quadruped results; weak Valkyrie survival
SAC	Off-policy, maximum entropy	Exploration and sample reuse in continuous control	Best Go1 and Valkyrie policies
TD3	Off-policy, deterministic	Tests whether twin critics reduce overestimation	Strong Go1 result; Valkyrie later diverged

2.3 From single environments to massively parallel simulation

Gym standardised the interface between an agent and an environment [6], while Gymnasium and Stable-Baselines3 provided maintained APIs and implementations on which the first EasyRTG CPU training lane was built [7, 8]. PyBullet provided a lightweight method for loading URDF models, inspecting joints and validating state calculations [9]. These tools were suitable for checking individual robot instances, but could not provide the volume of experience required by the final locomotion experiments. This limitation motivated the GPU training architecture described in Chapter 4.

2.4 Isaac Gym and the GPU-native turn

Makoviychuk et al. demonstrated that physics simulation and policy learning can remain on the GPU, removing the CPU transfer bottleneck and increasing throughput by orders of magnitude [10]. For EasyRTG, this approach enabled thousands of robot environments to run in parallel rather than one environment at a few hundred steps per second. The resulting throughput made repeated locomotion experiments feasible on a single workstation; the measured performance is reported in Section 6.4.

2.5 Orbit and Isaac Lab

Mittal et al. introduced Orbit, a modular framework for robot-learning environments based on Isaac Sim that was later developed into Isaac Lab [11]. Orbit demonstrates the structure that a well-designed environment framework can provide, but its Python environment definitions are intended for developers. EasyRTG uses Isaac Lab as its training substrate [12] while placing its configuration behind the desktop workflow. The underlying capability is retained without requiring the user to author an Isaac Lab environment directly.

2.6 Learning to walk in minutes

Rudin et al. trained thousands of simulated ANYmal robots to walk within minutes on a single GPU [13]. Their result established an important performance reference and motivated the initial use of ANYmal in this project. However, the reported speed relied on an environment, reward and curriculum designed specifically for that robot. EasyRTG required ten ANYmal attempts before one run reached 95% of the episode limit (Appendix D). This difference illustrates the current cost of supporting a general no-code workflow rather than a specialist robot implementation.

2.7 Reward design as the real bottleneck

A reward is an optimisation target rather than a neutral description of a task. An agent may therefore exploit any difference between the intended behaviour and the quantity that is measured. Potential-based shaping preserves an optimal policy under specific conditions [14], but practical locomotion rewards combine objectives and penalties for which this guarantee does not generally hold. Henderson et al. and Engstrom et al. further show that implementation details and evaluation choices can materially affect deep-reinforcement-learning results [15, 16].

This literature supports treating reward authoring as a correctness problem, not only as a configuration task. EasyRTG therefore compiles and audits reward expressions before GPU training begins. The evaluation also reports episode length and behavioural measures alongside reward. Section 6.7 provides a direct example of the underlying risk: a policy achieved a high galloping reward without performing a true gallop.

2.8 Eureka: language models writing rewards

Ma et al. demonstrated through Eureka that a large language model can generate reward code and refine it using training feedback, achieving human-competitive reward design across several tasks [17]. Eureka is the closest published precedent for the EasyRTG assistant, but the two systems make different safety trade-offs. Eureka generates executable reward code and searches over it evolutionarily. EasyRTG generates expressions over an approved state vocabulary; these expressions are compiled through an AST whitelist and audited on synthetic state ranges before they can be saved. EasyRTG therefore accepts less expressive freedom in exchange for inspectability and controlled tool use.

2.9 DrEureka: pushing language-assisted design toward hardware

DrEureka extended language-assisted reward design towards sim-to-real transfer by using a language model to generate both reward and domain-randomisation configurations [18]. This work also identifies an important boundary of EasyRTG. The present system stops at a recorded policy in simulation and does not implement domain randomisation or hardware deployment. These capabilities are therefore included in the future-work requirements in Section 8.2.

2.10 General-purpose simulators: Gazebo and Unity ML-Agents

Gazebo provides a general robot simulator with programmatic interfaces and extensive ROS integration [19], while Unity ML-Agents turns game-engine scenes into learning environments [20]. Gazebo offers broad robotics support but requires substantial setup. Unity ML-Agents provides flexible visual environments but requires engine-specific authoring. Both platforms are more capable than EasyRTG within their intended domains; however, the path from a robot model to a trained policy still requires code and framework-specific knowledge.

2.11 Unity RL Playground: the closest accessibility claim

The Unity RL Playground presented by Ye et al. is the closest published system to the accessibility objective of EasyRTG, providing one-click locomotion training for imported mobile robots [21]. EasyRTG differs by exposing a shared state vocabulary for reward authoring, performing a reward audit before training, and retaining a run archive for later examination. The published description of Unity RL Playground does not report reward authoring at the same level of detail, although its stated scope is narrower.

2.12 Where EasyRTG sits

The literature provides established learning algorithms, high-throughput simulators and methods for language-assisted reward generation. The remaining accessibility problem is their integration into a coherent workflow. EasyRTG addresses this gap by making robot structure, reward semantics, training controls and supporting evidence visible within one application. Its contribution is therefore the design and evaluation of this integration, including an account of where the abstraction succeeds and where expert judgement remains necessary.

Table 3. Positioning EasyRTG against representative robot-simulation and learning platforms.

Platform	Robot import	Training interface	Reward authoring	Primary trade-off
Gazebo/ROS	Rich model ecosystem	Programmatic	Controller/environment code	Broad robotics integration; substantial setup
Unity ML-Agents	Scene/component based	Unity + Python	Agent scripts and trainer config	Flexible visuals; engine-specific authoring
Isaac Lab	USD/URDF workflows	Python/configuration	Environment/reward code	High throughput; expert-oriented
Unity RL Playground	Imported mobile robots	One-click task workflow	Task framework	Closest accessibility goal; narrower reported scope
EasyRTG	URDF with inferred morphology	Desktop no-code flow	Audited expressions from user or assistant	Accessible integration; currently simulation-only

3 Research Methodology and Requirements

3.1 Design-science approach

A design-science approach was adopted because the research questions concern both the design of an integrated tool and its behaviour when applied [22]. Knowledge was developed by building and evaluating the EasyRTG artefact rather than through observation alone. The process was iterative: a barrier was identified, a mechanism was implemented, its behaviour was evaluated, and the design was refined. This process follows the stages proposed by Peffers et al.: problem identification, definition of objectives, design and development, demonstration, evaluation and communication [23]. Chapter 5 records how these stages were repeated during development.

3.2 Evidence sources

The evaluation used four evidence sources. First, 50 Git commits between 11 June and 14 August 2026 record the sequence of architectural changes. Second, the source code and automated tests establish which functions were implemented. Third, each Isaac run retains its configuration and telemetry, allowing the numerical results in Chapter 6 to be recomputed from the archive. Finally, recorded policy videos provide behavioural evidence that cannot be obtained from reward and episode statistics alone.

One inclusion rule was applied throughout the analysis: timestamped directories containing numeric reward telemetry were included as analysable runs, while directories without telemetry were retained as pre-telemetry failures. This rule explains why the run count in this report is higher than the count presented in the earlier project presentation.

3.3 Functional and non-functional requirements

The implementation was assessed against eight requirements. Each requirement is linked to evidence that can be inspected in the repository.

Table 4. Traceable project requirements.

ID	Requirement	Verification evidence
FR1	Load and inspect an arbitrary URDF robot	Robot panel, morphology tests, A1/ANYmal/Go1/Valkyrie
FR2	Create observations, actions, rewards and terminations without editing code	Config panels and expression editor
FR3	Let an assistant propose and apply configuration through tools	Agent toolbox and confirmation gates
FR4	Launch and monitor GPU-parallel training	Isaac worker and telemetry charts
FR5	Replay and compare completed policies	Evaluation worker, videos and run archive
NFR1	Reject invalid or unsafe reward expressions before launch	Compiler, audit and configuration tests
NFR2	Keep the desktop responsive during long-running work	Subprocess workers and polling/streaming
NFR3	Preserve run provenance	Saved config, telemetry, checkpoints and video

3.4 Experimental protocol and metrics

The run archive contains experiments conducted between 13 July and 18 August 2026 on a single RTX 5000. The analysable set contains 34 runs: eight A1 PPO, ten ANYmal PPO, four Go1 PPO, three Go1 SAC, two Go1 TD3, three Valkyrie PPO, three Valkyrie SAC and one Valkyrie TD3. This distribution is unbalanced. The comparisons in Chapter 6 are therefore reported as observed project results rather than as a statistically complete benchmark.

The primary measures were mean episode reward, mean episode length and environment steps per second. Task-specific measures included forward velocity, uprightness, lateral drift and flight phase. Episode length is reported as a fraction of the 1,000-step limit because it indicates whether the robot remained stable for the episode. Fleet summaries select the run

with the highest observed reward for each robot-algorithm pairing. This improves readability but introduces best-run selection bias, which is addressed in Section 7.3.

3.5 Software verification

Software verification was performed by running the backend test suite in a new temporary directory with third-party pytest plugin autoloading disabled. The result was 256 passing tests, one skipped test and no failures. The tests cover configuration phases, expression compilation, morphology, contact state, episode timing, Isaac bridge calculations, action authoring, spawn height and smoke behaviour. The Flutter command-line tool did not return in the available environment; consequently, no current claim is made about frontend analysis or test status. This limitation is recorded in Section 7.3.

4 System Design and Architecture

4.1 Architectural overview

The architecture must support thousands of parallel GPU environments while keeping the desktop responsive. EasyRTG separates the user interface from local orchestration to satisfy both requirements. Flutter provides the desktop interface [24]. FastAPI manages application state, exposes REST and streaming endpoints, supervises workers and records project state [25]. The LLM assistant has no privileged access and calls the same bounded tools used by the application. Simulation is divided into a lightweight inspection lane and a GPU training lane, preventing simple checks from consuming a training session.

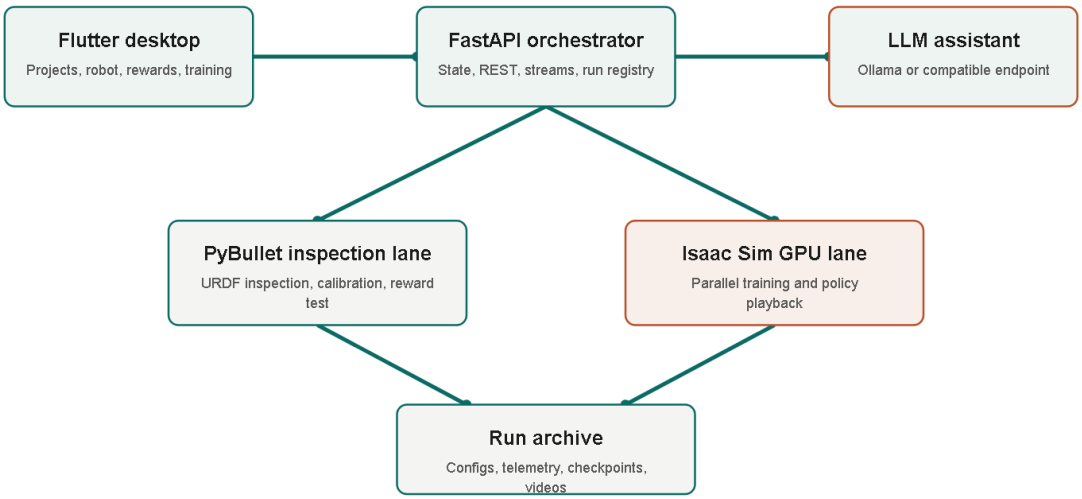


Figure 2. EasyRTG architecture, and the separation between the inspection and GPU training lanes.

The principal mechanisms follow the order of the workflow: the robot structure is inspected, a reward is defined against that structure, and the assistant then supports configuration and execution. This ordering ensures that assistant proposals are grounded in the inspected robot and the available state vocabulary.

4.2 URDF inspection and morphology

The first prototype inferred robot roles from link and joint names such as 'hip', 'foot' and 'ankle'. This approach failed when naming conventions changed. The resulting morphology layer instead treats the URDF as a kinematic tree. It identifies the base link, movable joints, leaf structures, candidate limbs and contact links from connectivity and joint properties. Standing height and spawn height are measured relative to the robot rather than copied from a sample model. Dimensionless signals such as `height_ratio` and `upright` can therefore retain the same meaning for quadruped and humanoid robots.

Morphology inference does not make every URDF automatically trainable. Poor inertial values, unrealistic joint limits and ambiguous contact geometry may still require calibration. Its contribution is narrower: a new robot is configured from structural evidence rather than from naming rules written for a previous model.

4.3 Expression rewards and the shared state vocabulary

After the robot structure has been derived, the required behaviour must be expressed as a reward. An early fixed catalogue allowed users to select only predefined components. Although simple, this approach restricted the range of behaviours and required application changes for every new task. EasyRTG instead represents each reward term as a mathematical expression over a shared state vocabulary, including `forward_velocity`, `upright`, `height_ratio`, `mean(contact)`, `flight_phase` and `episode_progress`. Vector widths are supplied by the morphology layer, so `mean(contact)` has the same interpretation whether the robot has two, four or six contact limbs.

Safety is provided through compilation rather than a restricted catalogue. Expressions are parsed into an abstract syntax tree and accepted only when every node is whitelisted. Attribute access and subscripting are excluded, preventing arbitrary Python execution. Configuration patches compile every enabled expression before saving. A separate audit evaluates reward

scale over synthetic state ranges and flags non-finite values, dominant penalties, all-penalty designs and sums that depend on the number of degrees of freedom. These checks cannot guarantee successful learning, but they detect several expensive failure classes before a GPU run begins.

4.4 Assistant and human oversight

The assistant makes expression-based configuration accessible to users who would not write the expressions directly. It can inspect robot details, calibrate a goal, patch a configuration, test a reward, start or stop training and inspect previous runs. Basic requests can be routed to a local Ollama model, while more complex goal interpretation can use an OpenAI-compatible endpoint. The system prompt provides the robot-specific vocabulary and available reward priors.

Tools that change configuration or control runs remain subject to explicit user approval. This design follows human-AI interaction guidance that recommends communicating system capabilities, making changes visible and preserving correction or undo [26]. The approval step introduces friction, but ensures that consequential actions remain reviewable and recoverable.

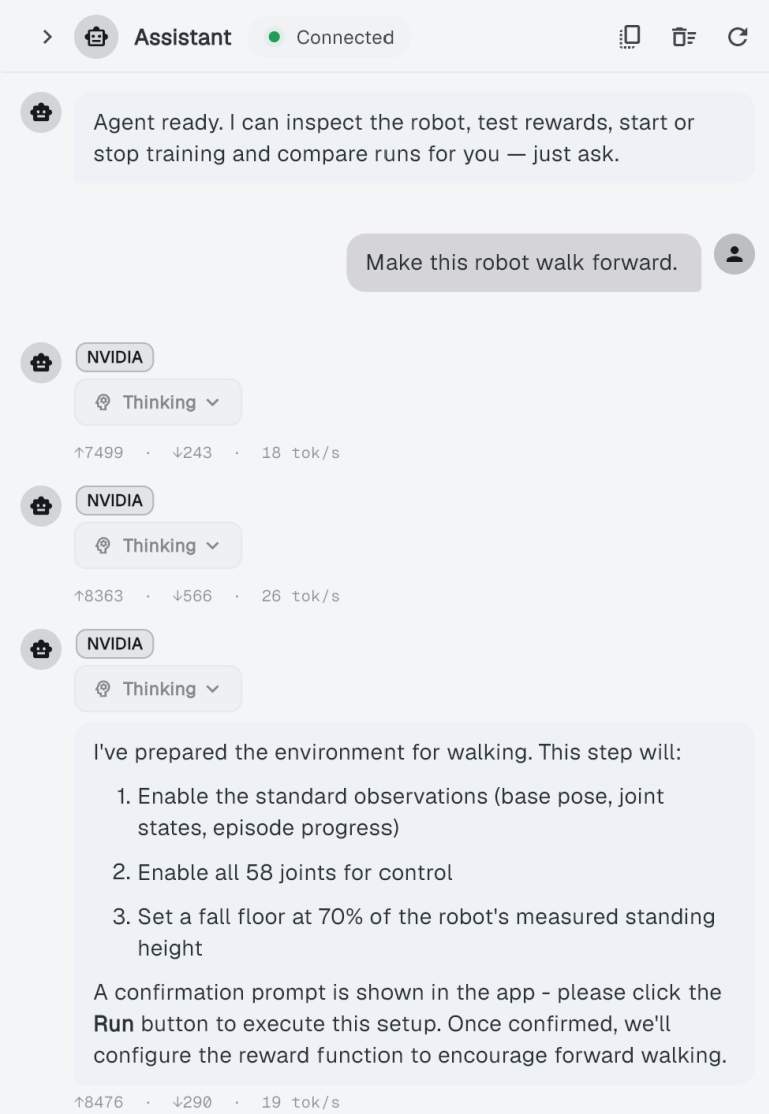


Figure 3. The EasyRTG desktop assistant, showing a generated training configuration and the changes offered for review.

4.5 Training, monitoring and evaluation

After a configuration is approved, Isaac training runs in a separate subprocess under Isaac Sim's Python environment and follows the installed Isaac Lab workflow [12]. The worker writes telemetry to a file, which the backend follows and streams to the Flutter charts. Every run retains its environment configuration, observation and action contracts, checkpoints, log and evaluation video. Live evaluation can launch NVIDIA's WebRTC client, while recorded evaluation runs headlessly and produces an MP4 file that can be replayed inside the desktop application.

Retained evidence is required because a reward curve cannot show whether a humanoid is walking forward, drifting sideways or merely remaining upright. The interpretation of several

experiments in Chapter 6 depends on the saved configuration, term-level telemetry and recorded video. Run provenance is therefore part of the evaluation mechanism rather than only a storage feature.

5 Implementation and Iterative Development

5.1 From proposal to working system

The original proposal described a six-week, CPU-first application based on PyBullet, Gymnasium and PPO from Stable-Baselines3. It was intended to run on an average laptop and to use predefined reward components with optional custom Python. The completed system differs from this plan in each area that introduced significant technical risk. Training moved to NVIDIA Isaac Sim on a GPU, custom Python rewards were removed, predefined reward components became audited expressions, and joint-name rules were replaced by morphology inference. The evaluation also expanded from PPO on sample robots to three algorithms across four robot families.

These changes do not indicate that the proposal failed. Instead, the prototype exposed requirements that could not have been established from the initial design alone. The following development history identifies the failures and observations that produced each major revision.

5.2 Milestone history

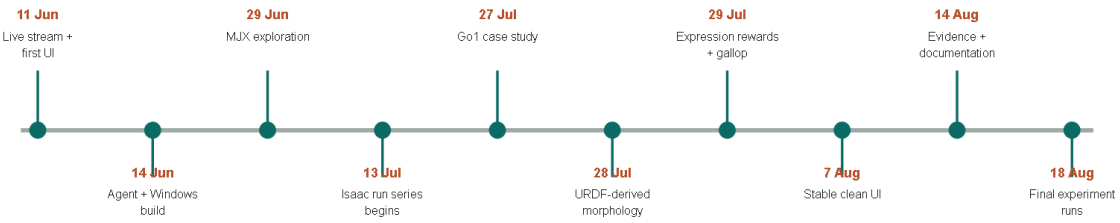


Figure 4. Major implementation and experiment milestones, reconstructed from Git dates and the run archive.

The commits dated 11 June, 'initial 300x300 30fps stream' and 'most of the app is working and is stable', established the first visible prototype. On 14 June, 'most of the agent is working' and 'Make Windows build/run work; stream JPEG sim frames' connected the assistant to a desktop

build. Although the commit messages are informal, this phase established one complete user interaction before the underlying components were refined.

The commit 'Add MJX backend, config revisions, and agent routing' on 29 June introduced an alternative high-performance backend. The following day's commit, 'switching to linux as mujoco isnt working good on win', records the operating-system difficulty that prevented this branch from becoming the final training lane. The attempt demonstrated that simulator and operating-system constraints remain present even when hidden by a desktop interface. EasyRTG subsequently adopted Isaac Sim as a separate GPU subprocess, allowing the main application to remain cross-platform while the training lane retained its required platform-specific runtime.

The locomotion case study began on 27 July with 'added unitree go1 robot as casestudy'. The first walking reward produced a low crawling behaviour because this was an easier way to gain forward velocity. The commit 'fix: reject crawling in walk goals' introduced robot-relative height termination to prevent this outcome. On 28 July, 'Derive robot structure from URDF, not joint names; add expression layer' removed a second source of robot-specific assumptions.

The most substantial redesign occurred on 29 July through four commits: 'Evaluate rewards as expressions on the Isaac lane', 'Give the Isaac lane real contact state, so a flight phase is expressible', 'Let the agent author the reward; delete the 4-keyword goal enum', and 'Delete the reward catalogue: expressions are the only reward path'. The final commit that day, 'Record the gallop result; gait is no longer inexpressible', captured the significance of these changes. Galloping cannot be defined by forward velocity alone because it requires a flight phase in which all four feet leave the ground. The task therefore required both Isaac contact state and a reward language capable of expressing that state. Before these mechanisms existed, the behaviour could not be specified by the application.

Later work focused on interface refinement and error handling. The 30 July commit 'Stop expressions written against observation names from being saved' followed an authoring error and added a further validation boundary. Interface refinement continued until the 7 August commit 'most stable and clean ui'. Commits on 14 August reorganised the architecture notes and updated the presentation and README. Additional run directories created on 18 August expanded the analysable dataset beyond the values reported in the earlier presentation.

5.3 Key engineering decisions

Six engineering decisions emerged from this sequence. Table 5 lists each retained decision, the alternative that was attempted and the reason for the final choice.

Table 5. Engineering decisions that emerged through iteration.

Decision	Alternative encountered	Reason retained
Two simulation lanes	Use PyBullet for everything	Fast inspection stays lightweight; Isaac provides training throughput
Expression rewards	Fixed catalogue or arbitrary Python	More behaviours than a catalogue; safer and more portable than exec
URDF-derived morphology	Joint-name heuristics	Supports unfamiliar naming and robot-relative measurements
Subprocess Isaac worker	Embed Isaac in FastAPI	Isaac uses its own Python and runtime, and can fail without taking the UI down
Human approval gates	Fully autonomous tool execution	Makes consequential changes visible and recoverable
Saved run directories	Ephemeral charts	Enables later recomputation, comparison and failure diagnosis

5.4 Security and reliability improvements

A no-code interface increases the user's dependence on application-level safeguards. Security and reliability are therefore part of the system design. An internal assessment identified unsafe pickle loading, arbitrary custom reward execution, ungated assistant changes and permissive CORS settings. The highest-priority issues were addressed during July. In particular, the custom-Python reward path was removed rather than protected through an expanding denylist; the expression compiler now excludes dangerous syntax by construction. Training, tuning and evaluation workers also check for conflicting jobs, and the desktop launcher will terminate only the backend process that it started.

5.5 Role of generative AI in the development process

Claude accelerated prototyping by translating my interface specifications into Dart code, drafting parts of the backend and helping diagnose failures. This support changed the speed of implementation, but not the allocation of responsibility. I determined what the application should do, selected the architecture, defined the interfaces and validation rules, integrated the generated code and decided which suggestions to retain. Git commits record the resulting sequence of changes, while the automated tests, run telemetry and evaluation videos show how proposed implementations were checked against the working system.

OpenAI Codex was introduced only after the application and evaluation evidence had been produced. It was used to restyle my original report material and to improve document formatting. It did not contribute to the EasyRTG application code, run the experiments or determine the findings. This separation is important because the software-development evidence in this chapter records my decisions and Claude-assisted implementation, whereas the final wording of the report is substantially Codex-assisted.

6 Experiments, Results and Findings

6.1 Run coverage and failure history

The earlier project presentation summarised 30 runs up to 3 August 2026, while the README later reported 32. A final scan for this report identified 40 timestamped run directories. Thirty-four contained numeric reward telemetry and six contained none. The difference reflects experiments added during the final fortnight, including most of the SAC and TD3 coverage. All fleet-level results in this chapter are based on the 34 analysable runs, which are listed individually in Appendix D.

Twelve runs maintained at least 95% of the episode limit. Several achieved only partial stability, and one Valkyrie TD3 run produced usable telemetry before a numerical failure. The chronological view in Appendix C shows that successful policies appeared only after repeated failures involving spawning, termination, reward scale and learned behaviour. Final evaluation videos therefore represent only part of the development evidence.

6.2 Fleet-level results

Table 6. Best observed run for each robot and algorithm pairing.

Robot	Algorithm	Best reward	Episode length	% of cap	Peak steps/s
A1	PPO	4,767.0	967.5	96.8%	32,056.8
ANYmal	PPO	769.8	954.6	95.5%	21,055.5
Go1	PPO	2,073.9	962.7	96.3%	27,500.0
Go1	SAC	2,250.1	1000.0	100.0%	27,840.3
Go1	TD3	2,112.7	990.9	99.1%	28,975.6
Valkyrie	PPO	276.1	204.2	20.4%	14,280.0
Valkyrie	SAC	1,283.8	821.0	82.1%	11,171.6
Valkyrie	TD3	287.0	206.2	20.6%	14,388.8

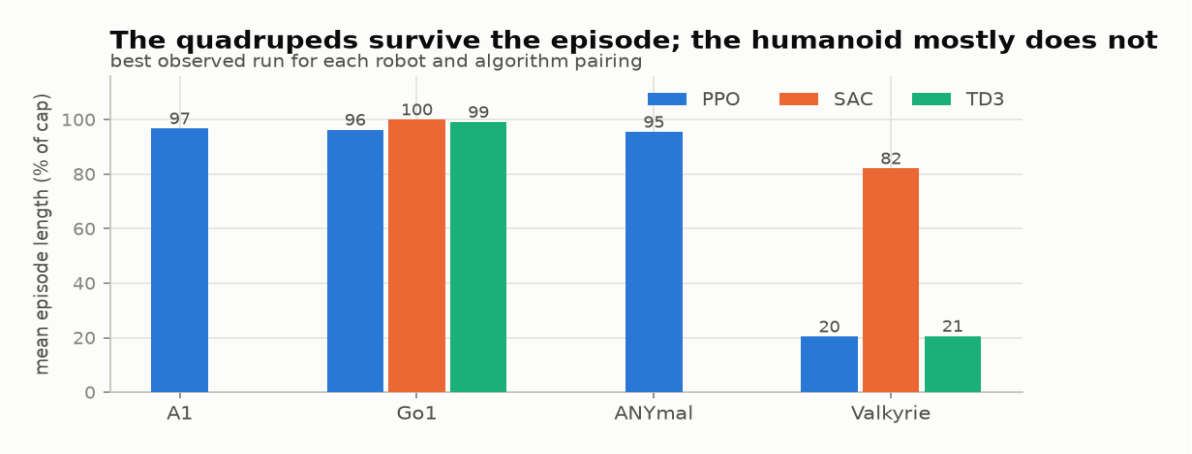


Figure 5. Episode survival for the best observed run in each robot and algorithm pairing.

The quadruped policies achieved the highest episode survival. A1 under PPO reached 967.5 steps and produced the highest reward in the archive, 4,767.0. ANYmal under PPO reached 954.6 steps. Go1 reached 962.7 steps under PPO, the full 1,000 under SAC and 990.9 under TD3. The Valkyrie results were lower and differed by learner: PPO reached 204.2 steps, SAC reached 821.0 and TD3 reached 206.2.

Interpretation. Reward functions differ between robots and tasks, so reward totals are not comparable across rows. A1's reward of 4,767.0 does not establish that it learned better than Go1; its reward expression assigns a different value per step. The `% of cap` column provides the directly comparable measure of episode survival.

6.3 What the reward actually paid for

Because reward totals are not comparable across robots, the more informative question is which terms produced a policy's score. EasyRTG logs every reward term separately, allowing the total to be decomposed at the point of highest observed performance.

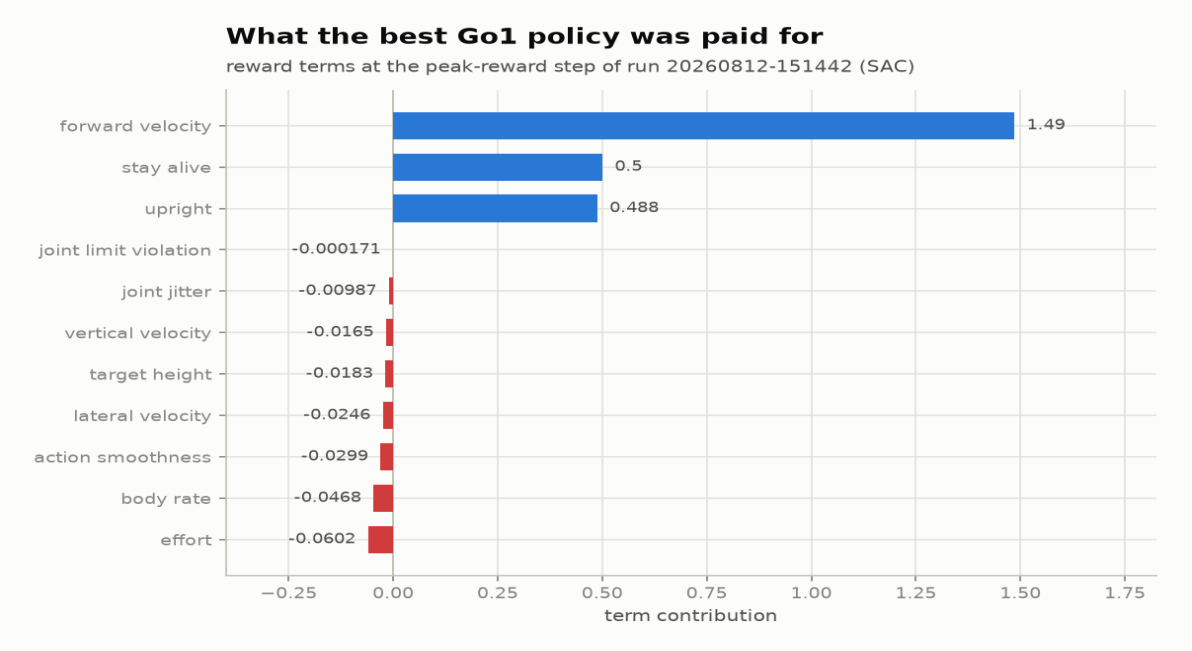


Figure 6. Term-by-term decomposition of the reward at the peak-reward step of the best Go1 run.

Three positive terms dominate the decomposition: forward velocity at 1.49, staying alive at 0.50 and uprightness at 0.49. The penalties are between one and four orders of magnitude smaller. Effort is the largest penalty at -0.060, while joint-limit violation is effectively zero at -0.00017, indicating that the policy was not operating against its joint limits.

Interpretation. The decomposition confirms that the policy earned its score primarily by moving forward while upright. It also shows that the penalties provided little constraint. A reward dominated by positive terms remains vulnerable to the type of misspecification demonstrated in Section 6.7, supporting the audit mechanism described in Section 4.3.

6.4 Throughput and the GPU lane

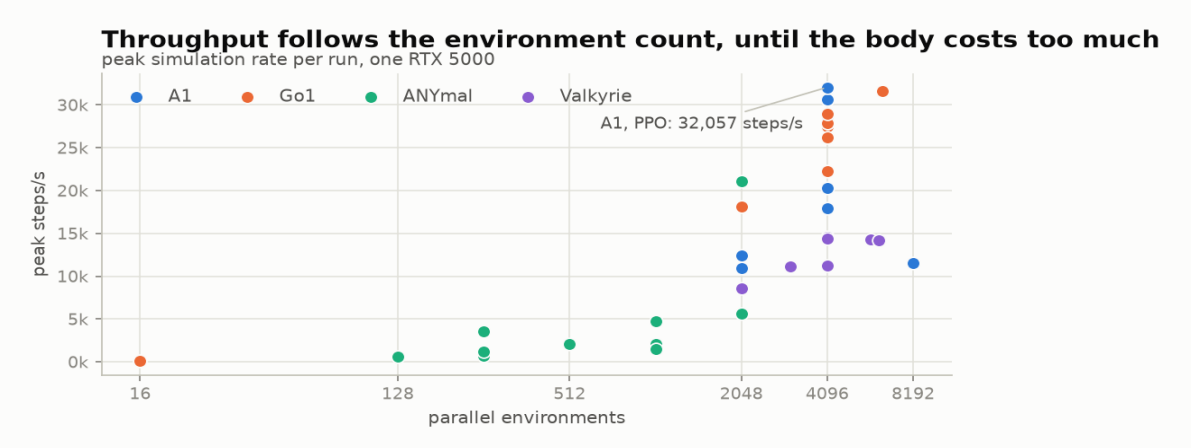


Figure 7. Peak simulation throughput against the number of parallel environments, coloured by robot.

The highest measured throughput was 32,056.8 environment steps per second, achieved by A1 at 4,096 environments. Go1 also exceeded 30,000 steps per second in recorded runs, while Valkyrie remained below approximately 14,600 steps per second at every tested batch size.

Increasing the number of environments provides more parallel work, but robot morphology determines the cost of each simulated step. A humanoid with more joints and bodies is therefore not directly comparable with a quadruped. RQ4 is explained by the interaction of two factors: parallelism determines the available workload, while robot complexity sets the practical throughput ceiling. The two diagnostic runs at 16 environments both remained below 100 steps per second, approximately three hundred times slower than the same robots at 4,096 environments.

6.5 How long learning took

Experiment cost depends on both simulation throughput and the number of steps required for learning. Learning time was measured as the number of environment steps before the smoothed mean episode length first remained above 900. This threshold indicates sustained stability rather than a single favourable logging interval.

Exactly twelve runs crossed this threshold, matching the twelve runs that reached 95% of the episode limit through a separately calculated measure. Appendix C provides the complete ordering. The best ANYmal run crossed after 2.51M steps, while the slowest successful Go1 run required 17.60M steps. Faster stabilisation did not imply a higher final reward. The highest-reward A1 run (20260728-202311; reward 4,767.0) required 16.42M steps to stabilise, whereas an earlier A1 run stabilised after 4.03M steps and achieved less than half that reward. Learning speed and final score must therefore be reported separately.

6.6 Go1: three learners, one solved robot

Go1 was the only robot evaluated with all three learners at the same environment count, making it the most suitable three-algorithm comparison.

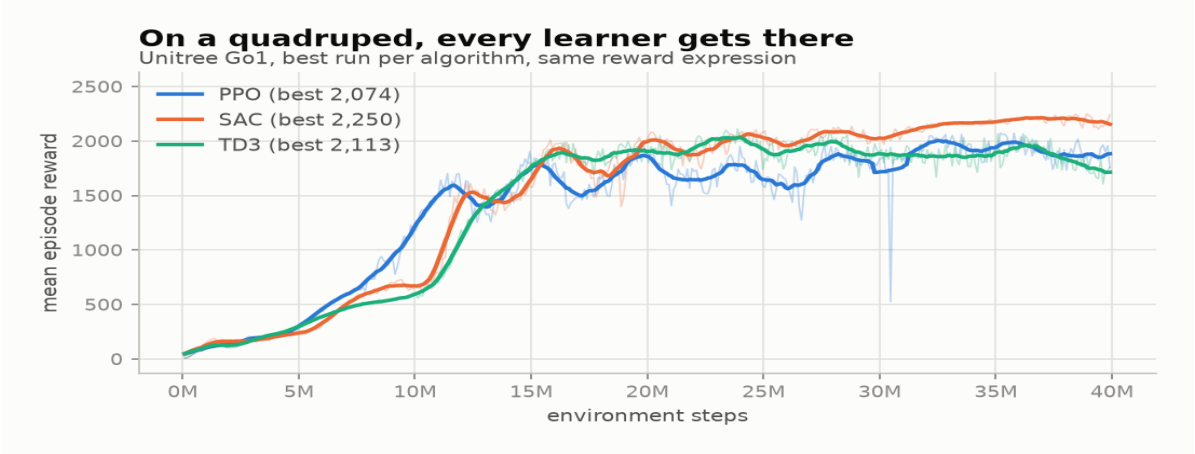


Figure 8. Reward for the best Go1 run under each learner, at 4,096 environments with the same reward expression.

All three Go1 policies converged to within 8% on reward: SAC reached 2,250.1, TD3 reached 2,112.7 and PPO reached 2,073.9. Episode survival was also similar at 100%, 99.1% and 96.3% of the limit respectively. The traces differ mainly in shape. SAC improves in distinct stages, PPO improves more smoothly, and TD3 rises fastest initially before flattening.

Interpretation. These results do not establish that the algorithms are equivalent. They show that the Go1 task was sufficiently tractable for all three learners to approach saturation. The comparison provides a baseline in which the pipeline does not visibly favour one learner. The more difficult Valkyrie task in Section 6.8 provides the separating case.

6.7 Go1 walking and galloping

The Go1 experiments demonstrate how reward semantics define learned behaviour. The walking reward encouraged forward motion, uprightness and sustained ground contact. The galloping reward additionally required a flight phase in which all four feet left the ground. In the presentation experiments, the walking policy travelled at 0.98 m/s, achieved an uprightness of 0.99 and survived 963 steps. The galloping policy reached 1.38 m/s, survived the full episode and was airborne for 43% of the stride. The robot and training budget were unchanged; only the reward semantics differed.

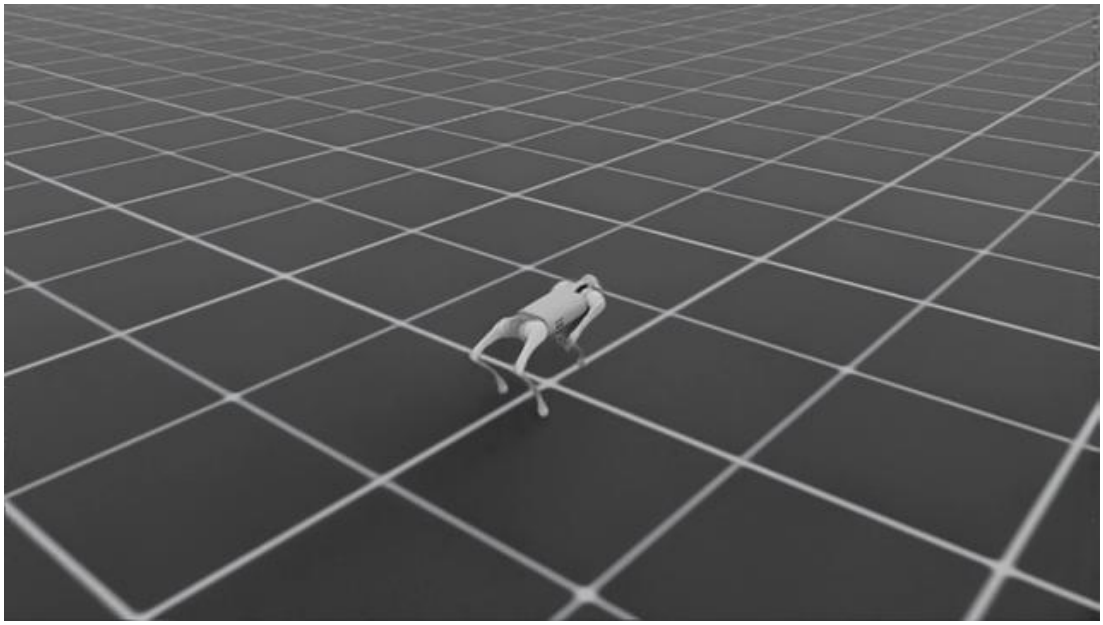


Figure 9. The Unitree Go1 during recorded policy evaluation.

The three-algorithm comparison exposed a limitation in the galloping reward. TD3 achieved the highest velocity, approximately 1.65 m/s, but had the lowest flight fraction at 29%. SAC was slower at 1.52 m/s but achieved a 52% flight fraction. TD3 had therefore learned a fast trot that scored well without producing the intended gallop.

Interpretation. This outcome is the reward-misspecification failure predicted by the literature [14, 15]. It demonstrates why contact-state measurements and video inspection must accompany reward. A high reward alone is insufficient evidence that the intended task has been learned.

6.8 Valkyrie: a controlled algorithm comparison

The strongest controlled comparison in the archive is between SAC and TD3 on Valkyrie. Both runs use the same reward expression and a 40M-step budget, holding the tested configuration constant except for the learner.

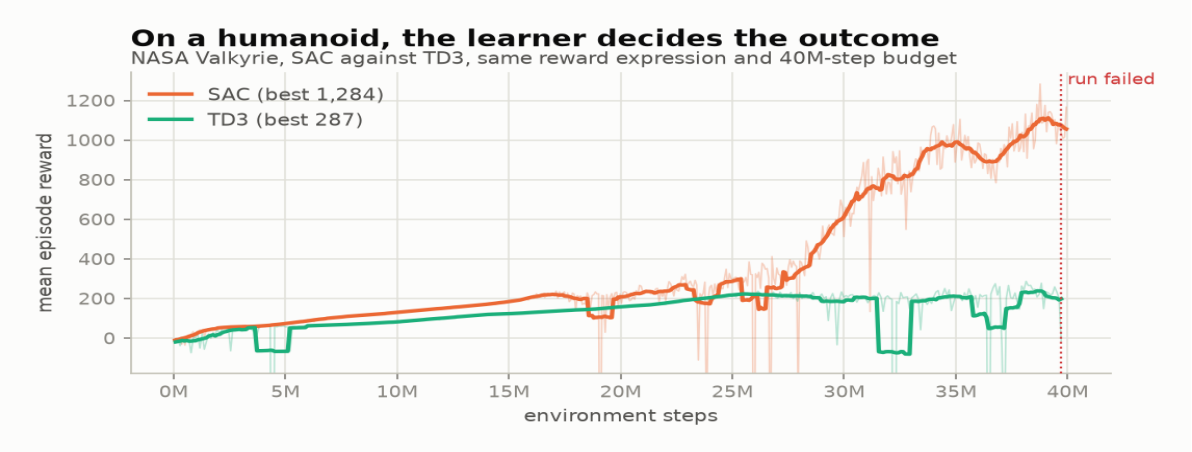


Figure 10. Reward for Valkyrie under SAC and TD3, same reward expression and same 40M-step budget.

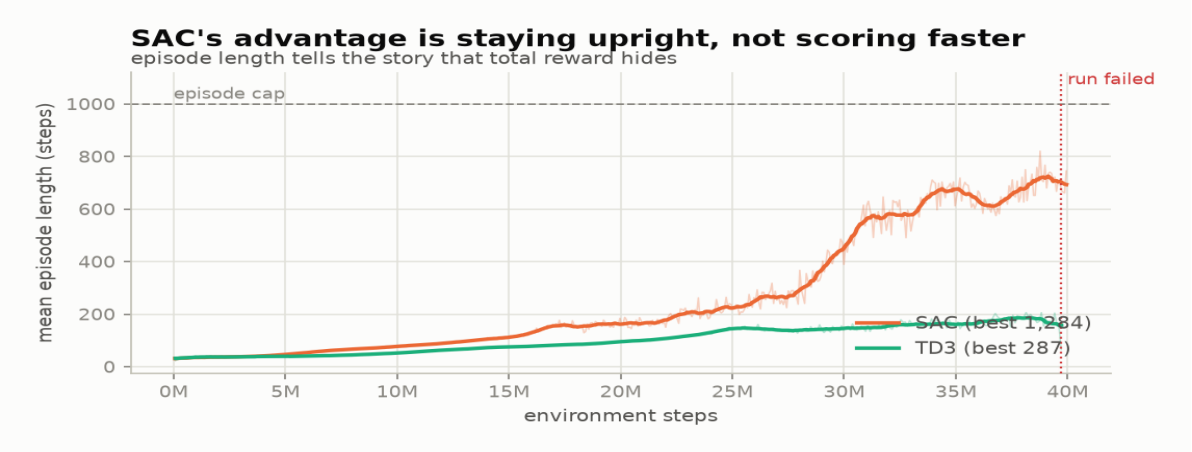


Figure 11. Episode length reveals that SAC's advantage came from staying upright, not from scoring faster.

Both learners initially improved and remained similar for approximately 20 million steps. Their behaviour then diverged. SAC reached a best mean reward of 1,283.8 with a mean episode length of 821.0 steps. TD3 remained near 200 steps of survival, reached a best reward of 287.0 and later stopped after producing NaN observations. The dotted line in the figures marks the point of failure.

At their best checkpoints, the policies received broadly similar reward per step. SAC achieved a larger total because it accumulated these terms for approximately four times as many steps before falling. Its advantage was therefore episode survival rather than a higher scoring rate. TD3 telemetry also showed several reward terms increasing by orders of magnitude before failure, indicating critic or numerical instability rather than only a failure to move forward.

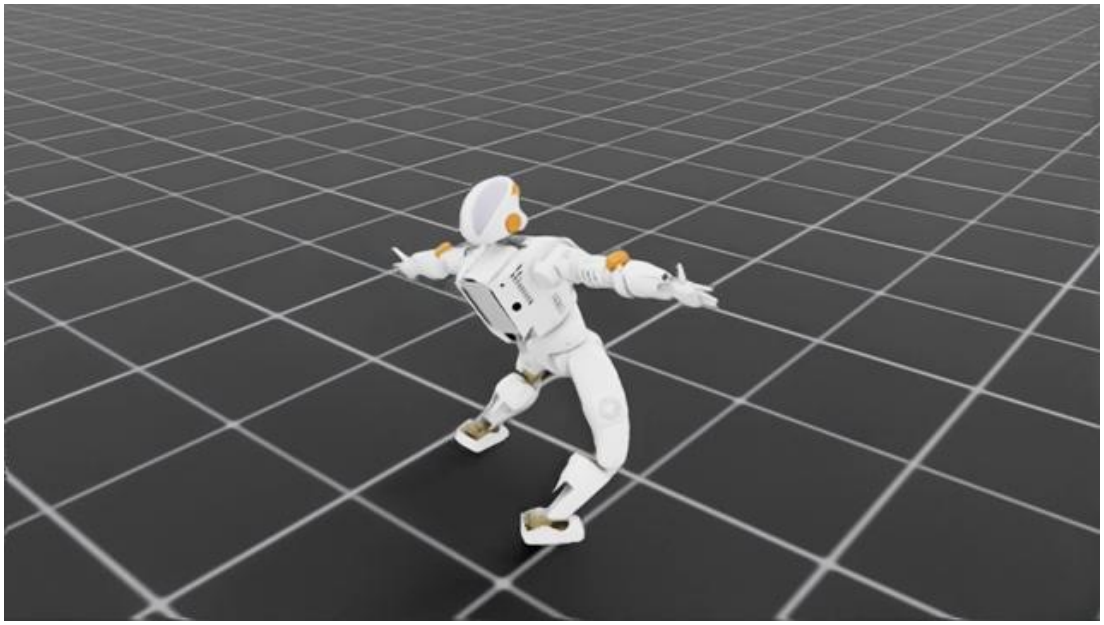


Figure 12. The NASA Valkyrie during recorded SAC policy evaluation.

Interpretation. Learner choice materially affected the Valkyrie outcome under the tested configuration, with SAC producing substantially longer survival than TD3. This is a result for one task and configuration, not evidence that SAC is generally superior. The limits of the comparison are discussed in Section 7.3.

6.9 Functional verification

The 256 passing backend tests provide a second form of evidence. They do not demonstrate learned robot behaviour. Instead, they verify the mechanisms required to interpret the training results: morphology handling, action mapping, state vocabulary, expression guards, episode clocks, contact signals, Isaac bridge calculations, configuration phases, and failure handling.

Neither form of evidence is sufficient in isolation. A visually convincing gait does not guarantee configuration safety, and passing standard checks offers no way to determine whether the output produces useful locomotion or something fundamentally unstable. The galloping result from Section 6.7 demonstrates this clearly, it satisfied every software check yet produced an unintended gait pattern rather than a functional motion. This gap between automated verification and behavioural evaluation confirms that both are required at once.

7 Discussion

7.1 Answering the research questions

RQ1: configuration with human oversight. EasyRTG breaks configuration into structural checks, relative calibration, and reward writing against an approved vocabulary. Compiling and auditing occur automatically before any tool runs. Users do not edit the underlying environment source code at all. A comparative user study measuring time savings has not yet been conducted.

RQ2: applicability across robot bodies. Treating morphology and state terms as dimensionless values allows one pipeline to generate viable policies for the A1, ANYmal, and Go1 quadrupeds alongside the Valkyrie humanoid. This approach removes fixed geometric constraints that typically constrain standard simulation pipelines. The test results confirm it functions correctly across these specific URDF models. It cannot however support manipulators or wheeled robots without significant architectural changes.

RQ3: differences between learners. All three algorithms produced nearly identical performance caps when controlling the Go1 quadruped. PPO, SAC, and TD3 showed negligible variance in final reward values within that specific environment. The Valkyrie results tell a different story entirely. SAC became the only viable option for that platform while neither PPO nor TD3 achieved stable gait patterns. Uneven algorithm coverage and incomplete hyperparameter tuning weaken these conclusions.

RQ4: simulation throughput. Parallel environments allowed throughput to scale until robot physics created a hard ceiling. A single GPU processed 32,056 steps per second across 4,096 environments for the A1 configuration. Valkyrie performed below that threshold due to increased rigid-body solver latency from its mass and joint count. These figures align with the architectural shifts toward Isaac detailed in Section 5.2, which matches GPU simulation benchmarks published by Castano [10] and Garcia [13].

7.2 What the project learned about abstraction

The core design challenge involved determining exactly which layers of complexity must remain visible. Obscuring simulator installation failed to eliminate underlying hardware constraints or operating system dependencies; the MJX experiment demonstrated this when sandboxed execution could not isolate itself from host GPU drivers. Arbitrary Python allows unconstrained

state mutations that compromise the runtime, while a static reward catalogue avoids these security risks but remains insufficient for environments requiring complex logic. EasyRTG resolved this by exposing only the inputs that drive learning, observations, actions, and termination signals, while compiling the rest of the execution logic into a controlled backend.

Defining the precise scope of the assistant proved significantly more difficult. The system generates configuration files that must pass validation before execution, functioning strictly as a drafting tool within a framework of human oversight. Project findings rely on concrete metrics, specifically six pre-telemetry failures alongside dozens of short-lived policies. Language models simplify the initial setup phase but cannot replace expert diagnosis; investigation into the Valkyrie TD3 failure confirmed that automated tools miss critical physics errors, ultimately requiring manual inspection of the URDF file.

7.3 Limitations and threats to validity

- The testing matrix favors certain configurations over others.
- SAC and TD3 targeted the Go1 and Valkyrie platforms while A1 and ANYmal were limited to PPO alone, preventing any broad claims about cross-robot algorithm consistency.
- Fleet summaries highlight the single most successful run per pairing rather than tracking an average across random seeds.
- These metrics reflect peak observed capability rather than expected typical performance.
- Throughput tests relied on one RTX 5000, so those numbers cannot be treated as universal benchmarks for other hardware.
- Every policy was evaluated strictly in simulation, omitting domain randomization or physical transfer.
- The no-code argument stands as an architectural benefit at this stage rather than a verified fact; without user studies measuring task completion or errors, empirical validation remains unestablished.
- Training outcomes remain dependent on the URDF file itself because automatic inference cannot fix bad mass distribution or incorrect joint limits hidden inside it.
- Observations remained locked at seven keys despite flexible rewards; camera feeds, terrain mapping, ROS networking, and multi-agent support remain unimplemented.

- Final verification focused strictly on backend logic because the Flutter tool failed to run properly in the test environment, leaving frontend status completely unconfirmed.

7.4 Practical and academic significance

The practical contribution of EasyRTG is an integrated path from a robot file to a recorded policy, combining configuration, training and supporting evidence within one application. Its academic contribution is a case study of language-assisted reward design without unrestricted code execution. The heterogeneous run archive also demonstrates why reward, episode survival and task-specific behaviour must be evaluated together.

The project does not demonstrate that robot learning has become effortless. Its more defensible contribution is that important configuration decisions are made visible, reviewable and repeatable. When a decision is incorrect, as in the galloping reward, the retained evidence makes the error identifiable.

8 Conclusion and Future Work

8.1 Conclusion

This work establishes an integrated desktop platform enabling users to import a URDF robot, define a locomotion task in natural language, and train reinforcement learning policies without writing low-level simulator scripts. All five primary objectives outlined in Chapter 1 were satisfied.

The system relies on a Flutter frontend, a FastAPI-orchestrated backend, PyBullet-based parameter inspection, and training within the NVIDIA Isaac Sim simulator using GPU hardware to autonomously extract motion physics directly from the URDF tree. Rewards are captured and audited as evaluated mathematical expressions via an established vocabulary facilitated by an LLM operating under strictly confined user validation protocols. The complete application underwent analysis across thirty-four Isaac Sim runs utilising four distinct robot families and three continuous-control algorithms.

Three significant outcomes defined the experimental trials. Parallel execution on the GPU eliminated the CPU bottleneck for multi-platform learning, achieving a peak throughput of 32,056.8 environment steps per second at a single workstation level using an RTX 5000 GPU. For quadruped models such as the Go1, learner selection exerted minimally significant

influence over final outcomes; PPO, SAC, and TD3 achieved greater than ninety-six percent survival episodes and converged upon comparable reward scores. Conversely, algorithm choice directly dictated training viability on higher-dimensional platforms like the Valkyrie humanoid. SAC emerged as the solitary method maintaining stable balance throughout its simulation, reaching an eighty-two point one percent survival score while PPO and TD3 plateaued at twenty percent, with TD3 subsequently degrading into numerical instability.

The engineering process was fundamentally driven by mitigating failures encountered during iterative integration. Initial challenges necessitated the inclusion of robot-relative floor-falling terminations, while fragile rule-based naming conventions were discarded in favour of a robust kinematic tree traversal mechanism. Directories and safety concerns regarding arbitrary Python execution mandated an AST expression compiler, whereas galloping traversal conditions informed the explicit modelling of foot surface contact states within Isaac Sim.

A notable limitation arose during the Go1 galloping test, where imprecise reward specifications drove TD3 to emulate a rapid trot movement. This unintended locomotion mode accumulated exceptionally high scalar rewards, causing the platform to entirely omit required airtime during its motion phase. The experiment confirms that unconstrained total reward volumes serve insufficiently as indicators of actual movement behaviour; without monitoring contact signals, episode length, and preliminary videos, unregulated reward accumulation remains practically insignificant. Finally, while the internal AI assistant frequently achieved functional reward functions from natural language inputs, it nonetheless requires a significantly expanded validation set to guarantee robustness in complex engineering scenarios.

The application evolved from its initial CPU-based conceptualisation into a fully operational pipeline. This progression demonstrates that robotic locomotion, reward design, and GPU-assisted training can be consolidated into a single no-code desktop platform while maintaining strict user oversight.

8.2 Future work

The first priority is a controlled usability study. A comparison between the EasyRTG workflow and a documented code-based baseline should measure task completion time, configuration errors, requests for assistance and a validated usability scale. This study is required to evaluate the no-code claim empirically rather than infer it from the architecture.

Observation expressions and per-episode velocity commands should also be introduced together, allowing one policy to respond to changing instructions rather than a fixed goal. A campaign worker could then propose, audit and test reward revisions automatically while retaining the user-approval mechanism described in Section 4.4.

Further work should add terrain and sensor observations, improve automatic dynamics checks, evaluate more robots with balanced seeds and algorithm budgets, and measure throughput on additional GPUs. Hyperparameters should be studied through a reproducible optimisation framework such as Optuna [27] rather than through individual experiments alone. Finally, domain randomisation and hardware export are required before EasyRTG can claim a complete route from simulation to physical robot behaviour. DrEureka provides a relevant model for this stage [18].

Bibliography

[1] R. S. Sutton and A. G. Barto, Reinforcement Learning: An Introduction, 2nd ed., Cambridge, MA: MIT Press, 2018.

[2] J. Kober, J. A. Bagnell, and J. Peters, "Reinforcement learning in robotics: A survey," *International Journal of Robotics Research,* vol. 32, no. 11, pp. 1238-1274, 2013.

[3] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," arXiv:1707.06347, 2017.

[4] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *Proceedings of ICML, pp. 1861-1870*, 2018.

[5] S. Fujimoto, H. van Hoof, and D. Meger, "Addressing function approximation error in actor-critic methods," in *Proceedings of ICML, pp. 1587-1596*, 2018.

[6] G. Brockman et al., "OpenAI Gym," arXiv:1606.01540, 2016.

[7] M. Towers et al., "Gymnasium: A standard interface for reinforcement learning environments," arXiv:2407.17032, 2024.

[8] A. Raffin et al., "Stable-Baselines3: Reliable reinforcement learning implementations," *Journal of Machine Learning Research,* vol. 22, no. 268, pp. 1-8, 2021.

[9] E. Coumans and Y. Bai, "PyBullet, a Python module for physics simulation for games, robotics and machine learning," 2016-2021. [Online]. Available: https://pybullet.org/.

[10] V. Makoviychuk et al., "Isaac Gym: High performance GPU-based physics simulation for robot learning," arXiv:2108.10470, 2021.

[11] M. Mittal et al., "Orbit: A unified simulation framework for interactive robot learning environments," *IEEE Robotics and Automation Letters,* vol. 8, no. 6, pp. 3740-3747, 2023.

[12] NVIDIA, "Isaac Lab documentation," 2026. [Online]. Available: https://isaac-sim.github.io/IsaacLab/.

[13] N. Rudin, D. Hoeller, P. Reist, and M. Hutter, "Learning to walk in minutes using massively parallel deep reinforcement learning," in *Proceedings of CoRL, PMLR vol. 164, pp. 91-100*, 2022.

[14] A. Y. Ng, D. Harada, and S. Russell, "Policy invariance under reward transformations: Theory and application to reward shaping," in *Proceedings of ICML, pp. 278-287*, 1999.

[15] P. Henderson et al., "Deep reinforcement learning that matters," in *Proceedings of AAAI, vol. 32, no. 1*, 2018.

[16] L. Engstrom et al., "Implementation matters in deep policy gradients: A case study on PPO and TRPO," in *Proceedings of ICLR*, 2020.

[17] Y. J. Ma et al., "Eureka: Human-level reward design via coding large language models," arXiv:2310.12931, 2023.

[18] Y. J. Ma et al., "DrEureka: Language model guided sim-to-real transfer," in *Proceedings of Robotics: Science and Systems*, 2024.

[19] N. Koenig and A. Howard, "Design and use paradigms for Gazebo, an open-source multi-robot simulator," in *Proceedings of IEEE/RSJ IROS, pp. 2149-2154*, 2004.

[20] A. Juliani et al., "Unity: A general platform for intelligent agents," arXiv:1809.02627, 2020.

[21] L. Ye et al., "Unity RL Playground: A versatile reinforcement learning framework for mobile robots," arXiv:2503.05146, 2025.

[22] A. R. Hevner, S. T. March, J. Park, and S. Ram, "Design science in information systems research," *MIS Quarterly,* vol. 28, no. 1, pp. 75-105, 2004.

[23] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee, "A design science research methodology for information systems research," *Journal of Management Information Systems,* vol. 24, no. 3, pp. 45-77, 2007.

[24] Flutter, "Flutter desktop documentation," 2026. [Online]. Available: https://docs.flutter.dev/platform-integration/desktop.

[25] FastAPI, "FastAPI documentation," 2026. [Online]. Available: https://fastapi.tiangolo.com/.

[26] S. Amershi et al., "Guidelines for human-AI interaction," in *Proceedings of CHI, paper 3, pp. 1-13*, 2019.

[27] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," in *Proceedings of KDD, pp. 2623-2631*, 2019.

Appendix A GitLab Repository and Running the Project

A.1 Repository

GitLab project URL: https://git.cs.bham.ac.uk/projects-2025-26/sxc1742

The `backend` directory contains the FastAPI orchestration service, robot inspection, configuration services, reinforcement-learning logic, Isaac subprocess workers and assistant tools. The Flutter desktop application is located in `frontend/rtg-flutter-app`. Architecture notes, setup records and design plans are stored in `docs`, while `scripts` contains the Windows and Linux launchers, build tools and media generators. The `backend/runs` directory stores each experiment's configuration, telemetry, checkpoints and evaluation video and is the source of every numerical result in Chapter 6.

A.2 Minimum setup

- Install Python 3.12, the Flutter desktop tooling and the backend packages listed in `backend/requirements.txt`.
- For GPU training, install NVIDIA Isaac Sim 6.0.1 and Isaac Lab 3.0 as described in `docs/isaac_setup.md`.
- Create the Python environment using `uv` and install the backend requirements into it.
- Start both services with `scripts/dev_run.ps1` on Windows or `scripts/dev_run.sh` on Linux.
- Load a URDF, configure the goal and reward, run Test reward, then launch Isaac training from the desktop application.

A.3 Verification commands

Backend tests:

```
$env:PYTEST_DISABLE_PLUGIN_AUTOLOAD=1; .\.venv\Scripts\python.exe -m pytest
backend\tests\ -q --basetemp=.pytest_tmp
```

Frontend:

```
cd frontend\rtg-flutter-app; flutter analyze; flutter test
```

Report figures:

```
py -m uv run --python 3.12 --with matplotlib --with numpy python
scripts/make_report_figures.py
```

Report document:

```
.venv\Scripts\python.exe scripts\write_report.py
```

Appendix B Experiment Coverage

Table 7. Coverage of the 34 analysable Isaac runs by robot and algorithm.

Robot	PPO	SAC	TD3	Total analysable
A1	8	0	0	8
ANYmal	10	0	0	10
Go1	4	3	2	9
Valkyrie	3	3	1	7

Six timestamped run directories contained no numeric reward telemetry and are retained as pre-telemetry failures. The additional `p3gallop` directory does not follow the timestamped `*-isaac` naming convention and is excluded from fleet-level calculations. Figures are generated from the run archive by `scripts/make_report_figures.py`; the run directories remain the authoritative data source.

Appendix C Supplementary Figures

The following figures support the analysis in Chapter 6 and were placed in the appendix to keep the main body within the page limit. Each figure is generated from the same run archive as the figures in the main text.

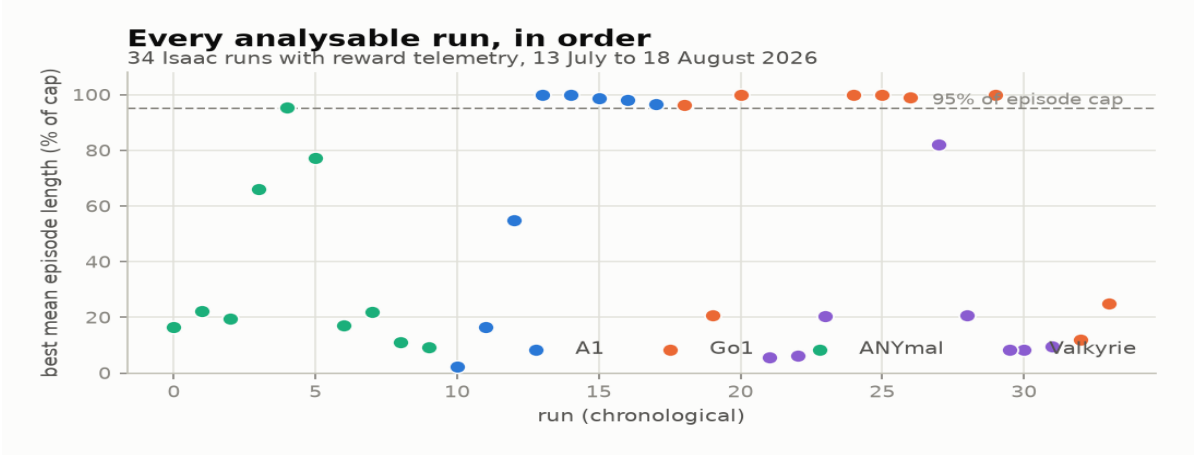


Figure 13. Best mean episode length for every analysable Isaac run, in chronological order.

The chronological view describes the development history rather than a single experimental outcome. Early ANYmal runs cluster at low episode lengths, while A1 and Go1 runs from late July onwards cluster near the episode limit. The two runs at 16 environments were added on 18 August as diagnostic experiments and establish the lower end of the throughput relationship discussed in Section 6.4.

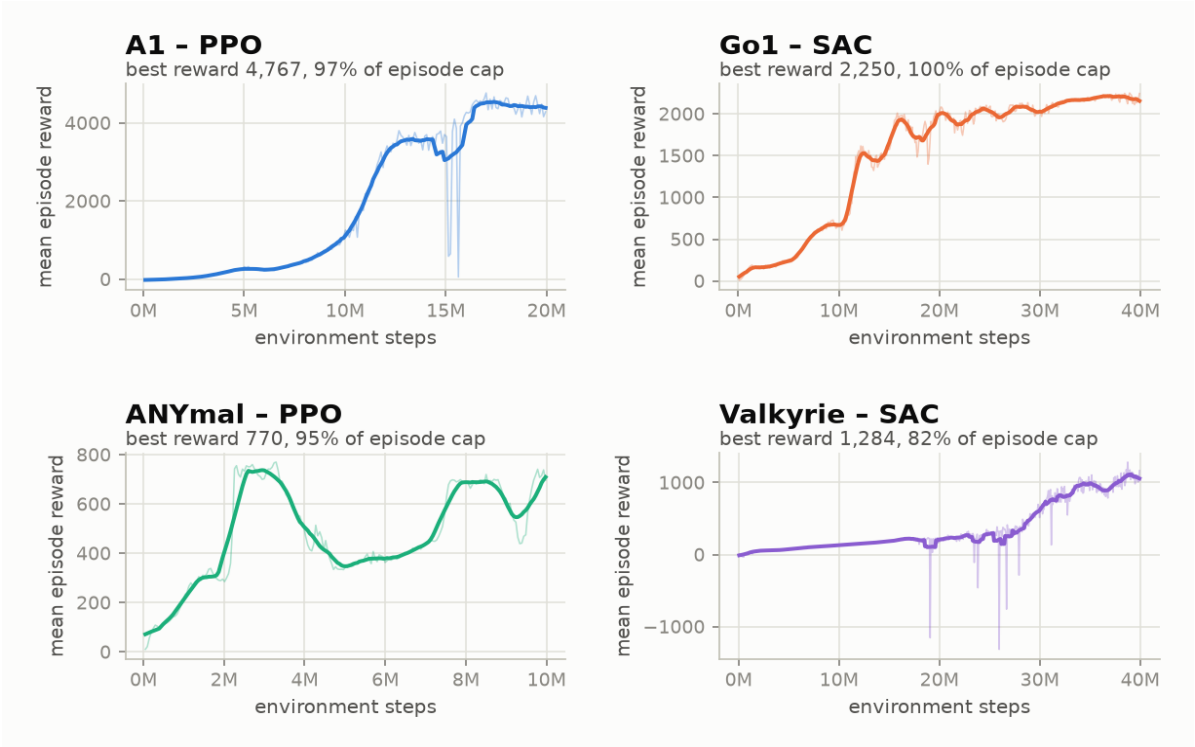


Figure 14. Reward against environment steps for the best run of each robot family.

The best run for each robot is plotted using the same scale within its panel. A1 improves rapidly and increases again near 16M steps, while Go1 improves in visible stages. ANYmal remains unstable and decreases by approximately half between 3M and 5M steps before recovering,

illustrating why ten attempts were required. Valkyrie improves slowly for 25M steps before accelerating; a shorter training budget would therefore have produced a different conclusion about humanoid learning.

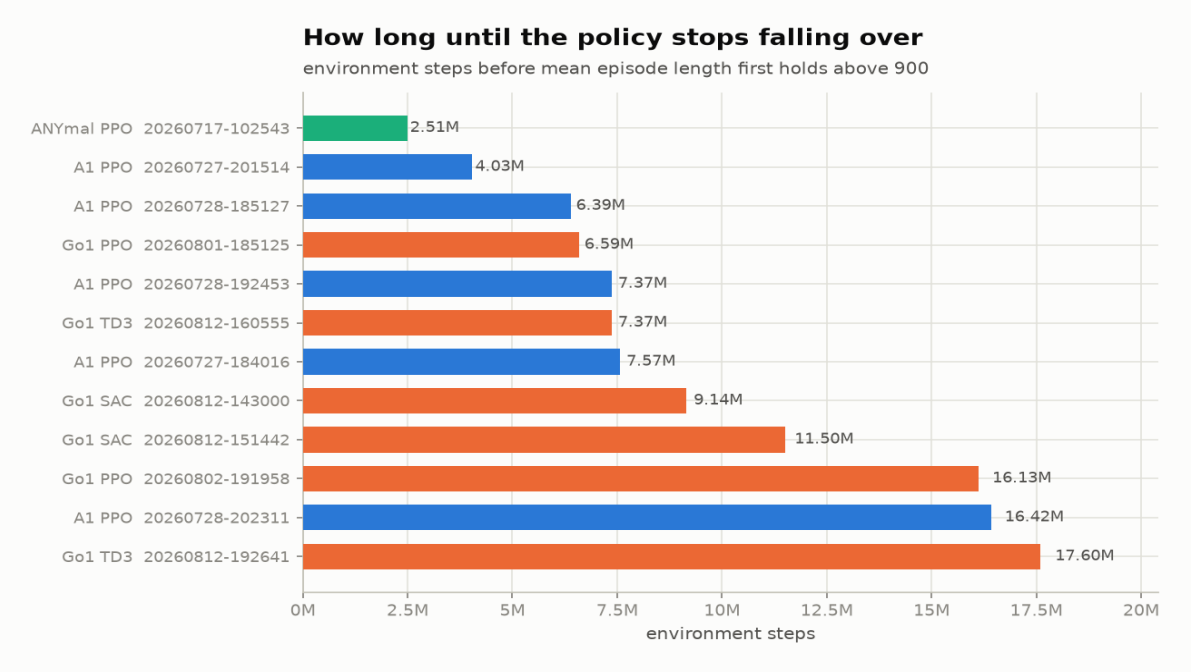


Figure 15. Environment steps before each run's smoothed mean episode length first holds above 900.

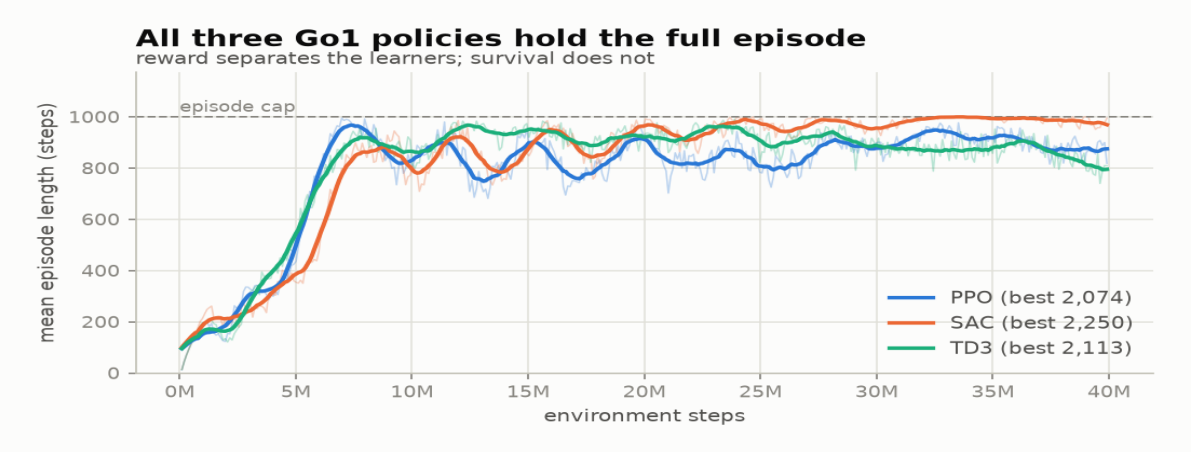


Figure 16. Episode length for the best Go1 run under each learner.

All three Go1 learners reach and maintain the episode limit, so episode survival does not distinguish them on this robot. In contrast, the Valkyrie comparison in Section 6.8 is separated primarily by survival.

Appendix D Full Run Index

Table D.1 lists every analysable run in the archive. Best reward is the highest observed mean episode reward. Episode length is the mean episode length at the same step, and peak steps per second is the highest recorded throughput. The fleet-level values in Chapter 6 are derived from this table.

Table 8. All 34 analysable Isaac runs, 13 July to 18 August 2026.

Run	Robot	Algorithm	Envs	Best reward	Episode length	% of cap	Peak steps/s
20260713-083121	ANYmal	PPO	512	137.0	165.9	16.6%	2,095
20260713-084227	ANYmal	PPO	256	193.4	222.2	22.2%	738
20260717-093639	ANYmal	PPO	256	159.1	195.4	19.5%	1,207
20260717-093932	ANYmal	PPO	256	518.0	659.4	65.9%	3,572
20260717-102543	ANYmal	PPO	2048	769.8	954.6	95.5%	21,056
20260717-110526	ANYmal	PPO	128	738.2	771.6	77.2%	591
20260722-133515	ANYmal	PPO	2048	144.0	170.5	17.0%	5,582
20260722-140347	ANYmal	PPO	1024	174.6	219.7	22.0%	2,047
20260722-141517	ANYmal	PPO	1024	93.1	112.0	11.2%	4,745
20260722-141803	ANYmal	PPO	1024	76.7	93.0	9.3%	1,525
20260727-173801	A1	PPO	2048	12.3	22.5	2.2%	12,420
20260727-180733	A1	PPO	2048	152.0	164.2	16.4%	10,966
20260727-182222	A1	PPO	8192	506.5	547.6	54.8%	11,493
20260727-184016	A1	PPO	4096	2,146.8	1000.0	100.0%	17,925
20260727-201514	A1	PPO	4096	2,338.3	1000.0	100.0%	20,285
20260728-185127	A1	PPO	4096	1,865.3	988.3	98.8%	30,599
20260728-192453	A1	PPO	4096	1,900.9	979.9	98.0%	28,462
20260728-202311	A1	PPO	4096	4,767.0	967.5	96.8%	32,057
20260801-185125	Go1	PPO	4096	2,073.9	962.7	96.3%	27,500

Run	Robot	Algorithm	Envs	Best reward	Episode length	% of cap	Peak steps/s
20260801-215400	Go1	PPO	2048	147.7	208.8	20.9%	18,081
20260802-191958	Go1	PPO	6400	1,600.6	1000.0	100.0%	31,631
20260803-211052	Valkyrie	PPO	4096	88.3	55.9	5.6%	14,556
20260803-230732	Valkyrie	PPO	4096	96.1	62.4	6.2%	11,230
20260803-233317	Valkyrie	PPO	5800	276.1	204.2	20.4%	14,280
20260812-143000	Go1	SAC	4096	1,952.9	1000.0	100.0%	26,180
20260812-151442	Go1	SAC	4096	2,250.1	1000.0	100.0%	27,840
20260812-160555	Go1	TD3	4096	2,112.7	990.9	99.1%	28,976
20260812-165024	Valkyrie	SAC	3048	1,283.8	821.0	82.1%	11,172
20260812-181127	Valkyrie	TD3	4096	287.0	206.2	20.6%	14,389
20260812-192641	Go1	TD3	4096	1,905.0	1000.0	100.0%	22,239
20260813-175701	Valkyrie	SAC	6200	251.3	82.8	8.3%	14,182
20260813-190946	Valkyrie	SAC	2048	140.0	94.1	9.4%	8,540
20260818-084821	Go1	SAC	16	97.5	121.1	12.1%	96
20260818-145041	Go1	PPO	16	147.2	250.4	25.0%	80

Twelve runs maintain at least 95% of the episode limit: five A1 PPO runs from 27 and 28 July, one ANYmal PPO run from 17 July at 2,048 environments, and all six Go1 runs at 4,096 environments or more. No Valkyrie run reaches this threshold.

Appendix E Compute Environment and Reproduction Notes

All training and evaluation results were produced on one workstation. The pre-release status of the simulator stack must be considered when reproducing the experiments.

Table 9. Compute and software environment used for all experiments.

Component	Version or specification
GPU	One NVIDIA RTX 5000
Isaac Sim	6.0.1-rc.7
Isaac Lab	3.0.0-beta2 (`v3.0.0-beta2.patch1`)

Component	Version or specification
Physics backend	Newton (replaces `omni.physics.tensors` from Isaac Sim 6.0)
Isaac Sim Python	3.12, bundled with Isaac Sim
Application Python	3.12, project virtual environment
Inspection simulator	PyBullet, headless
Backend	FastAPI
Desktop client	Flutter
Episode cap	1,000 steps

Isaac Sim is a release candidate and Isaac Lab is a beta. Isaac Sim 6.0 also replaces the earlier `omni.physics.tensors` implementation with the Newton physics backend. Results from an earlier build should therefore not be expected to match those reported here. Isaac Sim uses its own bundled Python and cannot run inside the application virtual environment, which is the practical reason for the subprocess architecture described in Section 4.5.

Three additional conditions apply to reproduction. First, throughput values are peak observations rather than sustained averages and vary with environment count. Second, the two 16-environment runs are diagnostic experiments rather than failed training attempts. Third, every figure is regenerated from `backend/runs` by `scripts/make_report_figures.py`; adding a run and executing the figure and document scripts updates the reported evidence without manual editing.